INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO NORTE
Campus Nova Cruz – Código INEP: 24083003
Av. José Rodrigues de Aquino Filho, Nº 640, RN 120, Alto de Santa Luzia, CEP 59215-000, Nova Cruz (RN)
CNPJ: 10.877.412/0015-633 – Telefone: (84) 4005-4107

QUESTÃO 1: Comparativo entre Estruturas Sequenciais (Estáticas/Dinâmicas) e Encadeadas em C++

Em termos de desempenho e gerenciamento de memória, compare a utilização de uma **Lista Sequencial Dinâmica (baseada em vetor)** e uma **Lista Simplesmente Encadeada (baseada em nós/ponteiros)**.

Em sua resposta, aborde obrigatoriamente os seguintes pontos:

1. **Acesso aos elementos:** Diferença no custo computacional ($O(1)$ vs $O(n)$) para acessar um elemento em uma posição arbitrária i.
2. **Inserção e Remoção:** Vantagens e desvantagens de cada estrutura ao inserir ou remover elementos no início, no meio e no fim.
3. **Overhead de Memória e Fragmentação:** Como a alocação dinâmica difere entre redimensionar um vetor (new[] / delete[]) e alocar nós individualmente (new No).

QUESTÃO 2: Integração de Estruturas de Dados Lineares em um Sistema de Navegação Web

Em um navegador web moderno, o gerenciamento da navegação de abas e do histórico de páginas exige a combinação de diferentes estruturas de dados lineares para garantir operações eficientes.

Considere duas funcionalidades essenciais:

1. **Histórico de Navegação de uma Aba (Botões Voltar e Avançar):** O usuário navega por novas páginas, pode clicar em "Voltar" para retornar à página anterior e em "Avançar" para ir para uma página desfeita. No entanto, se o usuário estiver em uma página anterior e acessar uma *nova* URL, todo o histórico de páginas do botão "Avançar" deve ser descartado.
2. **Fila de Download do Navegador (Gerenciamento de Requisições):** Os arquivos solicitados pelo usuário para download entram em uma fila e devem ser processados estritamente na ordem em que foram solicitados (primeiro a entrar, primeiro a sair).

Sua tarefa:

1. **Escolha e Justificativa das Estruturas:**
    - Quais são as **duas estruturas de dados lineares** mais adequadas para resolver, respectivamente, (a) o Histórico de Navegação (Voltar/Avançar) e (b) a Fila de Download?
    - Justifique tecnicamente a escolha de cada uma com base nas propriedades de acesso aos dados (LIFO, FIFO, etc.) e na complexidade temporal das operações ($O(1)$).

QUESTÃO 3: Pilha Encadeada em C++, Ponteiros e Prevenção de *Memory Leak*

Analise a implementação da classe Stack em C++ apresentada a seguir:

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
};

class Stack {
private:
    Node* top;

public:
    Stack() {
        top = nullptr;
    }

    void push(int value) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = top;
        top = newNode;
    }

    int pop() {
        if (top == nullptr) {
            std::cout << "Stack is empty!" << std::endl;
            return -1;
        }
        int value = top->data;
        top = top->next; // Linha Crítica
        return value;
    }

    ~Stack() {
        // O destrutor foi deixado vazio pelo desenvolvedor
    }
};
```

Com base no código acima, responda:

1. **Problema na remoção (pop):** Identifique o erro grave relacionado ao gerenciamento dinâmico de memória no método pop(). O que acontece com o nó desempilhado da memória Heap?
2. **Problema no destrutor (~Stack):** O que acontecerá na memória do computador quando um objeto da classe Stack for destruído (por exemplo, ao sair de escopo)?
3. **Desempenho e Corretude:** Qual seria a consequência do acúmulo dessa falha em uma aplicação do mundo real executada por longo período?

QUESTÃO 4: Fila Sequencial Circular com Vetor Dinâmico em C++

Um programador tentou implementar uma **Fila Circular Sequencial** utilizando vetor dinâmico. O método enqueue (inserção) funciona, mas ele não consegue fazer o redimensionamento do vetor dinâmico quando a fila enche.

Abaixo está o código parcial da classe Queue:

```cpp
#include <iostream>

class Queue {
private:
    int* elements;   // Ponteiro para o vetor dinâmico que armazena os elementos
    int capacity;    // Capacidade máxima atual do vetor dinâmico
    int front;       // Índice do elemento que está no início da fila
    int rear;        // Índice do elemento que está no fim da fila
    int count;       // Quantidade atual de elementos armazenados na fila

public:
    Queue(int initialCapacity = 4) {
        capacity = initialCapacity;
        elements = new int[capacity];
        front = 0;
        rear = -1;
        count = 0;
    }

    ~Queue() {
        delete[] elements;
    }

    bool isFull() const {
        return count == capacity;
    }

    // Trecho a ser corrigido/ampliado:
    void resize() {
        int newCapacity = capacity * 2;
        int* newElements = new int[newCapacity];

        // O desenvolvedor tentou copiar diretamente usando um loop simples:
        for (int i = 0; i < count; i++) {
            newElements[i] = elements[i]; // ERRO AQUI para filas circulares!
        }

        delete[] elements;
        elements = newElements;
        capacity = newCapacity;
        front = 0;
        rear = count - 1;
    }

    void enqueue(int value) {
        if (isFull()) {
            resize();
        }
        rear = (rear + 1) % capacity;
        elements[rear] = value;
        count++;
    }
};
```

Sua tarefa:

1. Explique por que a cópia direta newElements[i] = elements[i] falha em uma **fila circular** quando o ponteiro front não está na posição 0 (por exemplo, após algumas remoções e inserções).
2. Reescreva a implementação do método resize() de forma correta, garantindo que a ordem lógica dos elementos da fila seja preservada na nova área de memória alocada dinamicamente.

QUESTÃO 5: Aplicação de Pilha (Stack) na Validação de Sintaxe de Expressões

Em compiladores, interpretadores de linguagens de programação e calculadoras científicas, é essencial verificar se os delimitadores de uma expressão matemática ou bloco de código — como parênteses (), colchetes [] e chaves {} — estão corretamente balanceados e aninhados.

Uma expressão é considerada **válida** se:

1. Todo delimitador de abertura tem um delimitador de fechamento correspondente do mesmo tipo.
2. Os delimitadores são fechados na ordem exata e inversa em que foram abertos (propriedade LIFO — *Last In, First Out*).

Exemplos:

- "{[()()]}" → Válida
- "{[(])}" → Inválida (ordem de fechamento incorreta)
- "(((" → Inválida (delimitador de abertura não fechado)

Funções Auxiliares e Operações de Pilha Disponíveis:

Para manipular a sequência de entrada, assuma que você possui a seguinte função genérica:

- obterProximoCaractere(expressao): Lê e retorna o próximo caractere da expressao a cada chamada. Retorna o caractere especial FIM quando não houver mais caracteres a serem lidos.

Para manipular a pilha de suporte, você tem à disposição as seguintes operações da estrutura **Pilha de Caracteres**:

- P.push(c): Insere o caractere c no topo da pilha P.
- P.pop(): Remove e retorna o caractere do topo da pilha P.
- P.top(): Retorna o caractere do topo da pilha sem removê-lo.
- P.isEmpty(): Retorna verdadeiro se a pilha estiver vazia ou falso caso contrário.

Sua tarefa:

Elabore um **pseudocódigo** da função verificarBalanceamento(expressao) que recebe uma expressao e, utilizando a função obterProximoCaractere e as operações de pilha descritas acima, retorne verdadeiro caso a expressão esteja com os delimitadores balanceados e falso caso contrário.

QUESTÃO 6: Lista Duplamente Encadeada com Ponteiro de Cauda (tail) e Inserção Ordenada em C++

Considere uma **Lista Duplamente Encadeada** que armazena números inteiros em ordem crescente. A estrutura possui ponteiros para o primeiro nó (head) e para o último nó (tail).

As estruturas base foram definidas da seguinte forma:

```
struct Node {
    int value;
    Node* prev;
    Node* next;

    Node(int val) : value(val), prev(nullptr), next(nullptr) {}
};

class DoublyLinkedList {
    private:
        Node* head;
        Node* tail;

    public:
        DoublyLinkedList() : head(nullptr), tail(nullptr) {}
        ~DoublyLinkedList();

        // Método a ser implementado por você:
        void insertOrdered(int val);
};
```

Sua tarefa:

Implemente exclusivamente o método insertOrdered(int val) em C++. O método deve:

1. Alocar dinamicamente um novo nó com o valor val.
2. Inserir o nó na posição correta da lista, mantendo a ordem crescente dos elementos.
3. Tratar adequadamente todos os cenários possíveis:
    - Inserção em uma **lista vazia** (atualizando head e tail).
    - Inserção no **início da lista** (o novo valor é menor que head->value).
    - Inserção no **fim da lista** (o novo valor é maior ou igual a tail->value).
    - Inserção no **meio da lista** (entre dois nós existentes).
4. Atualizar corretamente todos os ponteiros adjacentes (prev e next), bem como head e tail quando necessário.

QUESTÃO 7: Tomada de Decisão em Projetos de Software (Trade-offs de Desempenho e Memória)

Para cada um dos cenários reais descritos abaixo, selecione a estrutura de dados **mais adequada** dentre as opções listadas e **justifique tecnicamente** a sua escolha com base em:

- Desempenho das operações (complexidade $O(1)$ vs $O(n)$);
- Eficiência do uso de memória e alocação.

Opções disponíveis para escolha:

- Lista Sequencial Estática em Vetor
- Lista Sequencial Dinâmica em Vetor
- Lista Simplesmente Encadeada
- Lista Duplamente Encadeada Circular
- Pilha (Sequencial ou Encadeada)
- Deque (Fila de Dupla Entrada)

Cenários:

1. **Cenário A — Sistema de Impressão de Documentos:**

Um servidor de impressão recebe solicitações de impressões vindas de vários computadores de uma rede. Os documentos devem ser processados exatamente na ordem em que chegaram. Além disso, o operador do sistema precisa, eventualmente, cancelar o documento que acabou de ser inserido no final da fila ou remover um documento com prioridade máxima do início.

2. **Cenário B — Histórico do Botão "Desfazer" (Undo/Redo) de um Editor de Texto:**

Um editor precisa armazenar os comandos executados pelo usuário para permitir que ele desfaça a última ação realizada ($Ctrl + Z$) ou refaça uma ação desfeita ($Ctrl + Y$). A operação de desfazer sempre afeta a ação mais recente.

3. **Cenário C — Módulo de Sensor de Temperatura em Sistema Embarcado de Baixo Custo:**

Um microcontrolador possui apenas 2 KB de RAM e coleta a temperatura de um ambiente a cada 1 segundo. Ele deve manter em memória um histórico fixo das últimas 60 medições para calcular a média móvel. O número de medições nunca ultrapassará 60 e o sistema precisa garantir tempo de execução estritamente determinístico, sem risco de falhas de alocação de memória dinâmicas durante a execução.

QUESTÃO 8: Lista Simplesmente Encadeada Circular em C++

Em uma **Lista Simplesmente Encadeada Circular**, o ponteiro next do último nó não aponta para nullptr, mas sim de volta para o primeiro nó (head). Quando a lista possui um único nó, o ponteiro next desse nó aponta para ele mesmo.

Considere a seguinte estrutura de classe em C++:

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;

    Node(int val) : data(val), next(nullptr) {}
};

class CircularLinkedList {
private:
    Node* head; // Aponta para o primeiro nó da lista circular

public:
    CircularLinkedList() : head(nullptr) {}
    ~CircularLinkedList();

    // Método a ser implementado por você:
    bool remove(int value);
};
```

Sua tarefa:

Implemente o método bool remove(int value) em C++. O método deve buscar e remover a **primeira ocorrência** de value na lista circular e liberar a memória do nó removido dinamicamente com delete.

Sua implementação deve tratar corretamente os seguintes casos:

1. **Lista vazia:** Retorna false.
2. **Remoção de nó em uma lista com apenas 1 elemento:** Se o valor for encontrado, libera o nó, atualiza head = nullptr e retorna true.
3. **Remoção do nó apontado por head (em lista com 2 ou mais nós):** Atualiza o ponteiro next do último nó da lista para apontar para o novo head, atualiza o head, libera a memória e retorna true.
4. **Remoção de um nó no meio ou fim da lista:** Atualiza o ponteiro next do nó anterior, libera a memória e retorna true.
5. **Valor não encontrado na lista:** Percorre a lista exatamente uma volta sem entrar em loop infinito e retorna false.

QUESTÃO 9: Deques (*Double-Ended Queues*) e Implementações (Vetor Dinâmico x Lista Duplamente Encadeada)

O Deque é uma estrutura linear versátil que estende as propriedades de pilhas e filas por permitir operações de inserção e remoção em ambas as extremidades (início e fim).

Com base nos seus conhecimentos sobre Deques e suas formas de representação em memória:

1. **Axiomas e Generalização:** Explique teoricamente como o Deque pode funcionar simultaneamente como uma **Pilha** (LIFO) e como uma **Fila** (FIFO), indicando quais pares de operações do Deque devem ser combinados para simular cada uma dessas estruturas.
2. **Comparativo de Overhead:** Compare o uso de memória entre um Deque implementado como **Vetor Dinâmico** e um Deque implementado como **Lista Duplamente Encadeada**. Qual das abordagens apresenta maior *overhead* de ponteiros por elemento armazenado e qual pode provocar maior perda por capacidade ociosa?

QUESTÃO 10: Vantagens, Desvantagens e Invariantes de Listas Duplamente Encadeadas e Circulares

As estruturas encadeadas oferecem grande flexibilidade de dimensionamento em memória, mas a escolha entre **Lista Simplesmente Encadeada**, **Lista Duplamente Encadeada** e **Lista Circular** impacta diretamente a complexidade do código e as propriedades de navegação da estrutura.

Com base nisso, responda às seguintes questões teóricas:

1. **Trade-off da Lista Duplamente Encadeada:** Qual é a principal vantagem teórica da Lista Duplamente Encadeada em relação à Simplesmente Encadeada ao realizar a remoção de um nó arbitrário já conhecido (dado um ponteiro para ele)? Qual é a contrapartida dessa escolha em termos de complexidade de código e alocação de memória?
2. **Invariantes e Casos de Borda na Lista Circular:** Em uma Lista Simplesmente Encadeada Circular, qual é a principal alteração na condição de parada ao percorrer toda a lista em comparação com uma lista encadeada convencional? Explique como a ausência de nullptr afeta a verificação do fim da estrutura e o risco de *loops* infinitos.